Smoothing and Nonparametric Regression Part I

1 Smoothing — Big Picture

1.1 What is smoothing?

Smoothing is a central idea in nonparametric statistics. The goal is to recover an unknown curve or function from noisy data.

In parametric models, we assume the curve belongs to a known family (e.g., normal density, linear regression). In smoothing, we avoid strong parametric assumptions and instead let the data determine the shape.

In these notes, the main curve estimation problems are:

- **Density estimation:** estimating an unknown density f from data.
- **Nonparametric regression:** estimating an unknown regression function $r(x) = \mathbb{E}(Y \mid X = x)$.

1.2 Why is the parameter in density estimation infinite dimensional?

In density estimation, we observe

$$X_1, \ldots, X_n \sim f,$$

where the density f is unknown.

Unlike parametric models, the parameter is not a finite vector. In parametric problems, we set $f = \text{Normal}(\mu, \sigma^2)$ or $f = \Gamma(a, b)$ etc. Then we have a fixed number of parameters. Once estimated, we know the whole density. The parameter is the entire function

$$f : \mathbb{R} \to \mathbb{R}.$$

Infinite-dimensional parameter

To understand why f is infinite dimensional, recall that the dimension of a space is the number of basis elements needed to describe its elements.

In parametric models:

$$\boldsymbol{\theta} \in \mathbb{R}^p,$$

so only p numbers determine the model. Note that $\boldsymbol{\theta} = \sum_{i=1}^{p} \theta_i \boldsymbol{e}_i$, where $\boldsymbol{e}_i$ is the unit-vector with 1 at *i-th* position and zero otherwise. Here $\{\boldsymbol{e}_i\}_{i=1}^{p}$ is the set of basis vectors for the Euclidean space.

For functions, we consider bases suitable for functional bases. In general, a function is expanded in a basis as:

$$f(x) = \sum_{k=1}^{\infty} \theta_k \phi_k(x),$$

where $\{\phi_k\}$ is, for example, a Fourier or spline basis or known parametric densities with different parameters.

A completely general smooth function requires infinitely many coefficients:

$$\boldsymbol{\theta} = (\theta_1, \theta_2, \theta_3, \dots).$$

There is no finite p such that all densities can be represented using only $\theta_1, \dots, \theta_p$. Therefore, the space of densities is *infinite dimensional*. The degree of smoothness is determined by these θ_i's.

Key insight: 1) Density estimation is fundamentally harder than parametric estimation because we are estimating an infinite-dimensional object. From a heuristic point of view, we need an infinite supply of data to learn an infinite-dimensional parameter. However, in a real data problem, we only have a limited amount of data. Thus, we need something extra. 'Smoothing' is that extra component. It is an assumption. So there is a trade-off between the required sample size needed for the estimation and the assumed smoothness.

2) Intuitively, we can only allow estimating more parameters if we have access to more data. A model is considered more complex if it needs more parameters for its characterization. Thus, more data allows estimation of models with greater complexity.

3) Another important aspect is to restrict the domain of the data. This has both computational and theoretical grounding. If the domain is restricted, we cannot control model complexity anymore from a theoretical point of view. Computationally, it prevents the generalizability of the fitted model. Specifically, we only have information about the function where we have observations. One needs to make stringent assumptions to predict things beyond that domain.

Two recurring motifs

Most smoothing methods share two fundamental design choices:

1. **How to define "closeness" (local neighborhood)** around the target point x. For example:
   - points within a window of width h,
   - points weighted by a kernel function,
   - nearest neighbors.
2. **What to do inside the neighborhood.** Once we identify nearby observations, we must decide how to combine them:
   - local averaging,
   - local linear regression,
   - higher-order local polynomial fitting.

Thus smoothing is always about balancing:

local information (variance) versus *global stability (bias)*.

2 Density Estimation

2.1 Setup

Let $X_1, \ldots, X_n$ be a random sample from a continuous distribution with:

- cumulative distribution function (CDF) F,
- probability density function (PDF) f.

The density f is unknown, and our goal is to estimate it from the observed data.

Density estimation is important because it provides:

- an overall summary of the distribution,
- identification of skewness, multimodality, and tail behavior,
- a foundation for clustering, anomaly detection, and Bayesian modeling.

2.2 Empirical CDF

A natural first estimator is the empirical distribution function:

$$\hat{F}_n(x) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}(X_i \leq x).$$

Key facts:

- $\hat{F}_n(x)$ is a step function with jumps of size $1/n$.
- By the Glivenko–Cantelli theorem,

$$\sup_x |\hat{F}_n(x) - F(x)| \to 0 \quad \text{almost surely.}$$

- The empirical CDF is nonparametric and requires no tuning parameters. (Sample size is the only factor here.)

R illustration:

```
library(NSM3)
data(discrepancy.scores)

plot(ecdf(discrepancy.scores),
     main = "Empirical CDF for spatial ability scores",
     xlab = "Score", ylab = "Empirical probability")
```

The ECDF is often the starting point for density estimation. But density estimation requires much finer control.

2.3 Histogram estimator

The histogram is the oldest and simplest density estimator.

Partition the real line into bins:

$$I_j = (c_j - h/2, c_j + h/2],$$

where:

- h is the bin width,
- c_j are bin centers.

The histogram estimator is:

$$\hat{f}(x) = \frac{\#\{i : X_i \in I_j\}}{nh}, \qquad x \in I_j.$$

Interpretation

- The numerator counts how many observations fall in the bin.
- Dividing by n gives an estimated probability mass.
- Dividing by h converts mass into density.

Choosing bin width

The most important tuning parameter is h.

- If h is too small: histogram is very rough (high variance).
- If h is too large: histogram oversmooths (high bias).

A common rule is Freedman–Diaconis:

$$h = 2 \cdot \mathrm{IQR}(X) \cdot n^{-1/3}.$$

This choice adapts to data spread and is robust to outliers.

R illustration:

```
hist(discrepancy.scores,
     freq = FALSE,
     breaks = "FD",
     main = "Histogram with FD bin width")
```

Histogram with ggplot

```
library(ggplot2)

cal.binwidth = function(x){
  2*(quantile(x, .75) - quantile(x, .25)) /
    (length(x)^(1/3))
}

binwidth = round(cal.binwidth(discrepancy.scores), 3)

ggplot(data.frame(scores = discrepancy.scores),
       aes(x = scores)) +
  geom_histogram(binwidth = binwidth,
                 fill = "white",
                 color = "black") +
  labs(title = "Histogram density estimate",
       x = "Score", y = "Density")
```

2.4 Kernel density estimation

Histograms suffer from discontinuities and dependence on bin placement. Kernel density estimation provides a smoother alternative.

The kernel density estimator (KDE) is:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - X_i}{h}\right).$$

Kernel interpretation

Each observation contributes a small "bump" centered at X_i:

- K determines the shape of the bump,
- h determines the width of the bump.

The estimate is the sum of all bumps.

Kernel conditions

A valid kernel satisfies:

$$K(x) \geq 0, \qquad K(x) = K(-x), \qquad \int K(x)\, dx = 1.$$

Common choices include:

- Gaussian kernel,
- Epanechnikov kernel,
- rectangular (uniform) kernel.

Bandwidth controls smoothness

The bandwidth h is the most important parameter:

- smaller $h \Rightarrow$ rougher estimate (low bias, high variance),
- larger $h \Rightarrow$ smoother estimate (high bias, low variance).

R example (rectangular kernel, fixed bandwidth):

```
density(discrepancy.scores,
        kernel = "r",
        bw = 1/(4 * sqrt(3)),
        n = 2^(14))

plot(density(discrepancy.scores,
             kernel = "r",
             bw = 1/(4 * sqrt(3)),
             n = 2^(14)),
     main = "Kernel density estimate")
```

Changing bandwidth using a built-in rule:

```
plot(density(discrepancy.scores,
             kernel = "r",
             bw = "nrd",
             n = 2^(14)),
     main = "KDE with NRD bandwidth")
```

2.5 Histogram as a kernel estimator

The histogram can be viewed as a kernel estimator using the rectangular kernel:

$$K(u) = \frac{1}{2}\mathbf{1}(|u| \leq 1).$$

Thus, the histogram is a special case of KDE.

2.6 Bias–variance trade-off (conceptual)

All smoothing estimators are governed by the bias–variance trade-off.

At a fixed point x:

$$\mathrm{MSE}(\hat{f}_h(x)) = \underbrace{\mathrm{Bias}^2(\hat{f}_h(x))}_{\uparrow \text{ with } h} + \underbrace{\mathrm{Var}(\hat{f}_h(x))}_{\downarrow \text{ with } h}.$$

As h increases:

- Bias increases,
- Variance decreases.

As h decreases:

- Bias decreases,

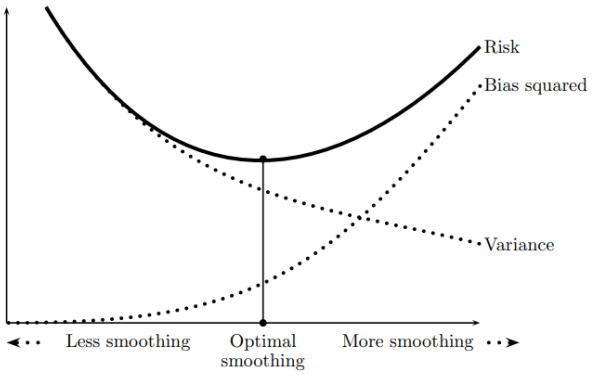

FIGURE 4.9. The bias–variance tradeoff. The bias increases and the variance decreases with the amount of smoothing. The optimal amount of smoothing, indicated by the vertical line, minimizes the risk = bias^2 + variance.

Figure 1: Wasserman, L. (2006). All of nonparametric statistics. New York, NY: Springer New York (Page 54).

- Variance increases.

The goal is to choose h to balance these two forces, often using:

- cross-validation,
- plug-in rules,
- likelihood-based methods.

2.7 Curse of dimensionality (high-level message)

Density estimation becomes dramatically harder in dimension $d > 1$.

In high dimensions:

- local neighborhoods contain very few points,
- variance increases quickly,
- the sample size needed grows exponentially with d.

This is known as the **curse of dimensionality**.

Practical mitigation strategies include:

- dimension reduction (PCA, manifold learning),
- variable selection,
- structural assumptions (additive models, sparsity),
- semiparametric modeling.

Key takeaway: Nonparametric smoothing is powerful in low dimension, but must be used carefully in high-dimensional problems.

3 Alternative Nonparametric Density Estimators

Kernel density estimation is the most widely used nonparametric density estimator, but several alternative approaches exist. These methods differ in how they control smoothness and how they adapt to the data.

3.1 k-Nearest Neighbor Density Estimation

Kernel density estimation uses a fixed bandwidth h for all locations. An alternative approach is to allow the smoothing region to depend on the local density of observations.

Let $R_k(x)$ denote the distance from x to its k-th nearest neighbor. The k-nearest neighbor density estimator is

$$\hat{f}(x) = \frac{k}{n} \frac{1}{\text{Volume of a } d\text{-dimensional ball with radius being } R_k(x)} = \frac{k}{n} \frac{1}{\frac{\pi^{d/2}}{\Gamma(\frac{d}{2}+1)} R_k(x)^d},$$

where

- d is the dimension,
- $R_k(x)$ is the distance to the k-th nearest observation.

Interpretation

- In regions where observations are dense, $R_k(x)$ is small, and the estimated density is large as we need to cross shorter distances to reach the *k-th* nearest point.
- In sparse regions, $R_k(x)$ is large, and the estimated density is small as we need to cross larger distances to reach the *k-th* nearest point.

Thus nearest-neighbor estimators automatically adapt to local density.

Bias–variance behavior

The tuning parameter is k.

- Small $k \Rightarrow$ low bias but high variance
- Large $k \Rightarrow$ high bias but low variance

Typical choices satisfy

$$k \to \infty, \qquad k/n \to 0.$$

R illustration

```
#For d=1
knn_density <- function(xgrid, x, k = 10){

 n <- length(x)

 sapply(xgrid, function(z){

   d <- sort(abs(x - z))
   rk <- d[k]

   k/(n*2*rk)
 })
}

x <- discrepancy.scores

grid <- seq(min(x),max(x),length=200)

plot(grid,
     knn_density(grid,x,k=10),
     type="l",
     main="k-NN density estimate",
     ylab="Density")
```

The k-nearest-neighbor density estimator can be viewed as a histogram-type estimator with variable bandwidth, where the bin width is chosen so that each bin contains exactly k observations.

$$h(x) = R_k(x).$$

More details can be found as: `https://faculty.washington.edu/yenchic/18W_425/Lec7_knn_basis.pdf`.

3.2 Adaptive Kernel Density Estimation

Standard kernel density estimation uses a fixed bandwidth h. Adaptive kernel estimators use location-specific bandwidths.

The adaptive kernel estimator is

$$\hat{f}(x) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{h_i}K\left(\frac{x - X_i}{h_i}\right),$$

where h_i depends on the local density near X_i.

Typical construction

One common approach is:

1. Compute a pilot density estimate $\tilde{f}(x)$. It is a preliminary estimate of the density, typically obtained using a standard kernel density estimator with a fixed bandwidth.

2. Define

$$h_i = h\left(\frac{g}{\tilde{f}(X_i)}\right)^{1/2},$$

where g is the geometric mean of $\tilde{f}(X_i)$ anf h is the global smoothing level.

This gives:

- small bandwidth in dense regions
- large bandwidth in sparse regions

R illustration

```
x <- discrepancy.scores

pilot <- density(x)

fhat <- approx(pilot$x,
               pilot$y,
               xout=x)$y

g <- exp(mean(log(fhat)))

h <- pilot$bw

adaptive_density <- function(grid){

 sapply(grid,function(z){

   hi <- h*(g/fhat)^(1/2)

   mean(dnorm((z-x)/hi)/hi)
 })
}

grid <- seq(min(x),max(x),length=200)

plot(grid,
     adaptive_density(grid),
     type="l",
     main="Adaptive KDE")
```

3.3 Orthogonal Series Density Estimation

Another approach is to approximate the density using an orthogonal basis expansion.

Suppose

$$f(x) = \sum_{j=0}^{\infty} \theta_j \phi_j(x),$$

where $\{\phi_j\}$ is an orthogonal basis.

The coefficients are estimated by

$$\hat{\theta}_j = \frac{1}{n} \sum_{i=1}^{n} \phi_j(X_i).$$

The density estimator is

$$\hat{f}(x) = \sum_{j=0}^{m} \hat{\theta}_j \phi_j(x).$$

Interpretation

The truncation parameter m controls smoothness:

- small m gives smooth estimates
- large m captures fine detail

This is analogous to bandwidth selection in KDE.

Example: Fourier basis

For data scaled to $[0, 1]$:

$$\phi_j(x) = \sqrt{2} \cos(\pi j x).$$

R illustration

```
x <- discrepancy.scores

xscaled <- (x-min(x))/(max(x)-min(x))

fourier_density <- function(grid,m=10){

 gridscaled <-(grid-min(x))/(max(x)-min(x))

 n <- length(xscaled)

 fhat <- rep(1,length(gridscaled))

 for(j in 1:m){

   theta <- mean(sqrt(2)*cos(pi*j*xscaled))

   fhat <- fhat +
```

```
            theta*sqrt(2)*cos(pi*j*gridscaled)
 }

 fhat/(max(x)-min(x))
}

grid <- seq(min(x),max(x),length=200)

plot(grid,
     fourier_density(grid,m=10),
     type="l",
     main="Orthogonal series estimate")
```

3.4 Log-Concave Density Estimation

Another way to estimate a density from univariate i.i.d. data is to impose simple shape assumptions such as monotonicity, convexity, or log-concavity. Unlike smoothing methods such as kernel density estimation or penalized regression, shape-constrained methods are fully automatic and do not require the choice of tuning parameters like a bandwidth or penalty constant.

Choosing tuning parameters is often difficult because the best values usually depend on unknown features of the true density. Shape-constrained methods avoid this difficulty by restricting the class of densities instead of introducing smoothing parameters.

Let f be a probability density on $\mathbb{R}$. The density f is called **log-concave** if it can be written in the form

$$f(x) = \exp\{\phi(x)\},$$

where $\phi : \mathbb{R} \to [-\infty, \infty)$ is a concave function and

$$\int_{\mathbb{R}} e^{\phi(x)}\, dx = 1.$$

Suppose $X_1, \ldots, X_n$ are independent observations from f. The log-concave density estimator is obtained by choosing the concave function ϕ that maximizes the average log-likelihood

$$\ell(\phi) = \frac{1}{n}\sum_{i=1}^{n} \log f(X_i) = \frac{1}{n}\sum_{i=1}^{n} \phi(X_i),$$

subject to the condition

$$\int_{\mathbb{R}} e^{\phi(x)}\, dx = 1.$$

The estimated density is then

$$\hat{f}(x) = \exp\{\hat{\phi}(x)\}.$$

Examples include:

- Normal distribution
- Exponential distribution

- Uniform distribution

The estimator is the maximum likelihood estimator:

$$\hat{f} = \arg \max_{f \text{ log-concave}} \sum_{i=1}^{n} \log f(X_i).$$

Advantages

- No bandwidth selection
- Automatically smooth
- Fully nonparametric within the class
- Every log–concave density is automatically unimodal.
- In the multivariate domain, it gets complicated.
- It is actually kernel density estimation without bandwidth.

R illustration

```
library(logcondens)

x <- discrepancy.scores

fit <- logConDens(x)

plot(fit,
     main="Log-concave density estimate")
```

More details can be found as: `https://cran.r-project.org/web/packages/logcondens/vignettes/logcondens.pdf`

3.5 Mixture Density Estimation

Flexible densities can be constructed using mixtures. This approach can be viewed as a form of basis expansion, where the basis functions are known probability densities, often taken from an exponential family.

A mixture density has the form

$$f(x) = \sum_{j=1}^{k} \pi_j f_j(x; \theta_j),$$

where

- $\pi_j \geq 0$ are mixing weights with $\sum_{j=1}^{k} \pi_j = 1$,
- $f_j(x; \theta_j)$ are component densities,

- the components are commonly chosen from exponential families, such as Gaussian or Gamma distributions.

Thus, mixture density estimation represents the unknown density as a linear combination of known density bases, with the parameters and weights estimated from the data.

Parameters are typically estimated using the EM algorithm.

Interpretation

Mixtures can approximate very complex densities:

$$f(x) \approx \sum_{j=1}^{k} \pi_j \phi_j(x).$$

R illustration

```
library(mclust)

x <- discrepancy.scores

fit <- densityMclust(x)

plot(fit,
     what="density",
     main="Mixture density estimate")
```

3.6 Comparison of Methods

Different estimators control smoothness differently.

Method	Tuning parameter	Smoothness control
Histogram	bin width	piecewise constant
KDE	bandwidth h	global smoothing
k-NN	k	local smoothing
Adaptive KDE	h_i	adaptive smoothing
Series	truncation m	global basis
Log-concave	none	shape constraint
Mixture	components k	flexible parametric

Summary:

- KDE is usually the default choice.
- Nearest-neighbor and adaptive methods perform better for heterogeneous densities.
- Series estimators are important theoretically.
- Shape-constrained estimators avoid tuning parameters.

3.7 Data-driven choice of smoothing parameters

All nonparametric density estimators require a tuning parameter that controls smoothness:

- bandwidth h in kernel density estimation,
- number of bases m in series estimators,
- number of mixture components k in mixture models,
- number of neighbors k in nearest-neighbor estimators.

These parameters are typically chosen using data-driven methods.

Cross-validation. A common approach is leave-one-out cross-validation, which selects the parameter that minimizes an estimate of prediction error. For KDE this often uses

$$\mathrm{CV}(h) = \int \hat{f}_h(x)^2 \, dx - \frac{2}{n} \sum_{i=1}^{n} \hat{f}_{h,-i}(X_i),$$

where $\hat{f}_{h,-i}$ is the estimator computed without observation i. The first part depends on the choice of kernel.

Plug-in rules. Plug-in rules approximate the theoretically optimal choice (e.g., for KDE, $h \propto n^{-1/5}$, for histogram $h \propto n^{-1/3}$) by estimating unknown quantities from the data.

Information criteria. For mixture models (and other likelihood-based density models), the number of components can be selected using criteria such as BIC:

$$\mathrm{BIC} = -2 \log L + p \log n,$$

where p is the number of free parameters.

Silverman's rule of thumb: A commonly used bandwidth choice for kernel density estimation is

$$h = 1.06 \, \hat{\sigma} \, n^{-1/5},$$

which provides a simple plug-in approximation to the optimal bandwidth under a normal reference model.

Examples

1. KDE bandwidth selection: built-in plug-in / rules-of-thumb.

```r
x <- discrepancy.scores

d_nrd0 <- density(x, bw = "nrd0")  # Silverman-type rule
d_nrd  <- density(x, bw = "nrd")   # Scott-type variant

plot(d_nrd0, main = "KDE: bw = nrd0 vs nrd")
lines(d_nrd)
legend("topright", lty = 1, legend = c("nrd0", "nrd"))
```

2. KDE bandwidth via likelihood cross-validation (LCV).

```
# install.packages("ks")  # if needed
library(ks)

x <- discrepancy.scores
# ks::hpi chooses a bandwidth by plug-in;
# ks::hlscv uses likelihood cross-validation
h_pi   <- hpi(x)     # plug-in bandwidth
h_lscv <- hlscv(x)   # likelihood CV bandwidth

d_pi   <- density(x, bw = h_pi)
d_lscv <- density(x, bw = h_lscv)

plot(d_pi, main = "KDE bandwidth: plug-in vs LSCV")
lines(d_lscv)
legend("topright", lty = 1, legend = c("Plug-in (hpi)", "LSCV (hlscv)"
   ))
```

3. Choosing number of mixture components by BIC.

```
# install.packages("mclust")  # if needed
library(mclust)

x <- discrepancy.scores
fit <- densityMclust(x)   # fits k=1,...,G and selects by BIC

fit$G                      # selected number of components
fit$BIC                    # BIC values across candidate G

plot(fit, what = "density",
     main = "Gaussian mixture density (BIC-selected)")
```

4. Choosing number of basis terms in a series estimator (simple CV).

```
x <- discrepancy.scores
n <- length(x)

# Rescale to [0,1] for Fourier basis
x0 <- (x - min(x)) / (max(x) - min(x))

phi <- function(t, j) sqrt(2) * cos(pi * j * t)

series_fit <- function(tgrid, x0, m){
  fhat <- rep(1, length(tgrid))  # start with constant term
  for(j in 1:m){
    theta <- mean(phi(x0, j))
    fhat <- fhat + theta * phi(tgrid, j)
  }
  pmax(fhat, 0)  # truncate negatives for plotting
}

# crude CV criterion: compare fitted CDF to ECDF on a grid
grid0 <- seq(0, 1, length.out = 300)
Fn0 <- ecdf(x0)
```

```r
cv_m <- function(m){
  fhat <- series_fit(grid0, x0, m)
  Fhat <- cumsum(fhat) * (grid0[2] - grid0[1])
  mean((Fhat - Fn0(grid0))^2)
}

ms <- 1:30
cv_vals <- sapply(ms, cv_m)
m_star <- ms[which.min(cv_vals)]

m_star
plot(ms, cv_vals, type="l",
     main="Series truncation chosen by CV proxy",
     xlab="m", ylab="CV criterion")
```

Takeaway: Bandwidth or model size is usually chosen from the data via plug-in rules, cross-validation, or information criteria to balance bias and variance.

3.8 Elbow method

A simple graphical approach for choosing smoothing parameters is the **elbow method**. The idea is to plot a measure of fit versus model complexity and select the point where improvements begin to level off.

Typical choices include:

- bandwidth h in kernel density estimation,
- number of basis functions m in series estimators,
- number of mixture components k in mixture models,
- number of neighbors k in nearest-neighbor estimators.

The selected value corresponds to the point where: additional complexity produces only small improvements. This point is called the **elbow** of the curve.

Advantages

- Simple and visual
- Requires minimal assumptions
- Useful for exploratory analysis

Limitations

- Subjective choice
- No formal optimality guarantees

R illustration: mixture components

```
fit <- Mclust(iris[,1:4], G = 1:10)

fit$modelName

bic <- fit$BIC[, grep(fit$modelName, colnames(fit$BIC))]

plot(1:10, -bic,
     type = "b",
     xlab = "Number of components",
     ylab = "BIC",
     main = "Elbow method for mixture size")
```

R illustration: basis size

```
x <- discrepancy.scores
x0 <- (x-min(x))/(max(x)-min(x))

phi <- function(t,j) sqrt(2)*cos(pi*j*t)

ISE <- function(m){

 grid <- seq(0,1,length=200)

 fhat <- rep(1,length(grid))

 for(j in 1:m){
   theta <- mean(phi(x0,j))
   fhat <- fhat + theta*phi(grid,j)
 }

 mean(fhat^2)
}

ms <- 1:30
vals <- sapply(ms,ISE)

plot(ms,vals,type="b",
     xlab="Number of basis terms",
     ylab="Criterion",
     main="Elbow method")
```

4 Bridge to Nonparametric Regression

4.1 Regression setting

Density estimation answers the question:

What is the overall distribution of a single variable X?

Regression answers a different but equally fundamental question:

How does the mean (or center) of Y change as a function of x?

Given n observed pairs (x_i, Y_i), a nonparametric regression model is:

$$Y_i = r(x_i) + \varepsilon_i, \qquad i = 1, \ldots, n, \qquad \mathbb{E}(\varepsilon_i) = 0.$$

Here:

- x_i is the predictor (covariate),
- Y_i is the response,
- $r(x)$ is the unknown regression function,
- ε_i represents noise or unexplained variation.

Interpretation under random covariates.
If the covariates are random draws from a distribution, then

$$r(x) = \mathbb{E}(Y \mid X = x).$$

Thus $r(x)$ represents the conditional mean curve of Y given $X = x$.

Main objective:

Estimate the regression function $r(\cdot)$ without assuming a specific parametric form.

This distinguishes nonparametric regression from classical linear regression:

$$Y_i = \beta_0 + \beta_1 x_i + \cdots + e_i,$$

where the functional form is fixed in advance.

Instead, nonparametric regression lets the data determine the shape of $r(x)$.

4.2 Regression smoothers: overview

Just as density estimation has histograms and kernel estimators, regression has a wide variety of smoothers.

The lecture highlights several important families:

- **Regressogram:** a histogram-type estimator for regression, using bin averages of Y values.
- **Running mean / local averaging:** estimate $r(x)$ by averaging responses near x.
- **Kernel regression (Nadaraya–Watson):** a weighted average where closer points receive higher weight.
- **Local regression / local polynomial fitting:** fit a low-degree polynomial in a neighborhood of x.

- **Spline regression and smoothing splines:** estimate $r(x)$ using flexible piecewise polynomials with smoothness penalties.
- **Wavelet smoothing:** represent $r(x)$ using multiscale basis functions (useful for irregular signals).

Although these methods differ in implementation, they share the same smoothing philosophy:

Estimate $r(x)$ using nearby observations, controlled by a tuning parameter.

5 Nonparametric Regression

5.1 Introduction and goals

Nonparametric regression is concerned with estimating the functional relationship between a response Y and predictor x when:

- the shape of the regression curve is unknown,
- we do not want to impose linearity,
- we only assume weak conditions on the errors.

This approach is especially useful when:

- exploratory plots suggest nonlinear trends,
- parametric models are too restrictive,
- robustness and flexibility are desired.

Focus of this lecture

Here we focus on **linear smoothers**.

A regression estimator $\hat{r}(x)$ is called a linear smoother if it can be written as:

$$\hat{r}(\boldsymbol{x}) = \sum_{i=1}^{n} w_i(\boldsymbol{x})\, Y_i,$$

where the weights $w_i(\boldsymbol{x})$ depend on:

- the target point $\boldsymbol{x}$,
- the design points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$,
- the bandwidth or smoothing parameter.

Thus, linear smoothers estimate $r(x)$ as a weighted average of observed responses. This is akin to the least squares estimate, where the prediction corresponding to a new $\boldsymbol{x}$ is $\hat{(y)} = \boldsymbol{x}^T\{(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\}\boldsymbol{y}$ which is $\boldsymbol{x}^T\{(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\}\boldsymbol{y}$. But we change $\boldsymbol{x}^T\{(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\}$ part.

5.2 Core regression model

We observe $(\boldsymbol{x}_i, Y_i)$ for $i = 1, \ldots, n$, and assume:

$$Y_i = r(\boldsymbol{x}_i) + \varepsilon_i, \qquad \mathbb{E}(\varepsilon_i) = 0.$$

The mean-zero assumption implies:

$$\mathbb{E}(Y_i) = r(\boldsymbol{x}_i).$$

So the regression function is the systematic signal, while ε_i represents random noise.

Homoscedasticity assumption

Sometimes we also assume constant error variance:

$$\mathrm{Var}(\varepsilon_i) = \sigma^2,$$

which does not depend on $\boldsymbol{x}$.

If instead $\mathrm{Var}(\varepsilon_i)$ changes with $\boldsymbol{x}$, the model is **heteroscedastic**, and smoothing methods may need modification.

5.3 Fixed design vs random design

Two common interpretations exist:

- **Fixed design:** the covariates $x_1, \ldots, x_n$ are treated as nonrandom, chosen by the experimenter.
- **Random design:** the covariates are random draws from a distribution, and the regression function satisfies

$$r(\boldsymbol{x}) = \mathbb{E}(Y \mid \boldsymbol{X} = \boldsymbol{x}).$$

In practice, the smoothing methodology is similar, but the theoretical interpretation differs.

5.4 Big picture: smoothing in regression

Nonparametric regression smoothers always involve:

- defining a neighborhood around $\boldsymbol{x}$,
- combining nearby Y_i values,
- controlling smoothness via a bandwidth or penalty parameter.

Just as in density estimation, smoothing is governed by:

Bias–variance trade-off.

- Too little smoothing $\Rightarrow$ noisy curve (high variance),
- Too much smoothing $\Rightarrow$ oversimplified curve (high bias).

Choosing the smoothing parameter is therefore one of the most important practical issues.

6 Linear Smoothers and the Smoothing Matrix

6.1 Definition

A central class of nonparametric regression estimators are called **linear smoothers**.

A smoother $\widehat{r}_n(\cdot)$ is a **linear smoother** if for each target point x there exist weights

$$\ell(\boldsymbol{x}) = (\ell_1(\boldsymbol{x}), \ldots, \ell_n(\boldsymbol{x}))^\top$$

such that

$$\widehat{r}_n(\boldsymbol{x}) = \sum_{i=1}^{n} \ell_i(\boldsymbol{x}) Y_i.$$

Thus, $\widehat{r}_n(\boldsymbol{x})$ is always a *weighted average* of the observed responses Y_i, where the weights depend on:

- the location $\boldsymbol{x}$,
- the design points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$,
- the smoothing parameter (bandwidth, binwidth, penalty, etc.).

Key point: Linearity refers to being linear in the observed responses Y_i, not linear in x.

6.2 Fitted values at the observed design points

Define the fitted regression values at the observed covariates $\boldsymbol{x}_i$:

$$\hat{\mathbf{r}} = \big(\widehat{r}_n(\boldsymbol{x}_1), \ldots, \widehat{r}_n(\boldsymbol{x}_n)\big)^\top.$$

Then we can write the entire fitted vector compactly as:

$$\hat{\mathbf{r}} = L\mathbf{Y},$$

where:

$$\mathbf{Y} = (Y_1, \ldots, Y_n)^\top,$$

and L is the **smoothing matrix** (also called the **hat matrix**).
The entries of L are:

$$L_{ij} = \ell_j(\boldsymbol{x}_i).$$

So L summarizes the entire smoother in matrix form.

6.3 Interpretation and properties

The smoothing matrix viewpoint is extremely useful because it provides:

- a unified way to describe many different smoothers,
- a direct route to bias–variance calculations,
- tools for inference, diagnostics, and degrees of freedom.

Row interpretation

- The ith row of L contains the weights applied to the data values $Y_1, \ldots, Y_n$ in order to compute the fitted value $\widehat{r}_n(\boldsymbol{x}_i)$.
- Thus, each row acts like an *effective kernel* centered at $\boldsymbol{x}_i$.
- Nearby points typically receive larger weight, while distant points receive small or zero weight.

Hat matrix analogy

In ordinary least squares regression:

$$\hat{\mathbf{Y}} = H\mathbf{Y},$$

where H is the classical hat matrix.
Similarly, in nonparametric regression:

$$\hat{\mathbf{r}} = L\mathbf{Y},$$

where L plays the same role but depends on the smoothing method.

Effective degrees of freedom

Many smoothers behave like regression models with a certain complexity. A common measure is:

$$\nu = \operatorname{tr}(L).$$

- If ν is small, the smoother is very smooth (low flexibility).
- If ν is large, the smoother is wiggly (high flexibility).

This is the nonparametric analogue of the number of parameters in a linear model.

Reproducing constants

A desirable property of smoothers is that they reproduce constant functions.
If $Y_i = c$ for all i, we would like:

$$\widehat{r}_n(\boldsymbol{x}) = c \quad \text{for all } \boldsymbol{x}.$$

A sufficient condition is:

$$\sum_{i=1}^{n} \ell_i(\boldsymbol{x}) = 1 \qquad \text{for all } \boldsymbol{x}.$$

This ensures the smoother is a proper local averaging procedure.

Remark: Many common smoothers (kernel regression, local linear regression, splines) satisfy this property.

7 Regressogram

7.1 Definition

The simplest regression smoother is the **regressogram**.

It is the regression analogue of the histogram estimator in density estimation.

Partition the covariate range $[a, b]$ into m disjoint bins:

$$B_1, \ldots, B_m,$$

each of equal width.
Let k_j denote the number of observations in bin B_j.
For any $x \in B_j$, define:

$$\widehat{r}_n(\boldsymbol{x}) = \frac{1}{k_j} \sum_{i:\, \boldsymbol{x}_i \in B_j} Y_i.$$

Thus:

- within each bin, the fitted curve is constant,
- the value is simply the sample mean of responses in that bin.

Linear smoother form

The regressogram can be written as:

$$\widehat{r}_n(\boldsymbol{x}) = \sum_{i=1}^{n} \ell_i(\boldsymbol{x}) Y_i,$$

where:

$$\ell_i(x) = \begin{cases} \frac{1}{k_j}, & \boldsymbol{x} \in B_j \text{ and } \boldsymbol{x}_i \in B_j, \\ 0, & \text{otherwise.} \end{cases}$$

So the regressogram is indeed a linear smoother.

7.2 Bandwidth and smoothness

The bin width is:

$$h = \frac{b - a}{m}.$$

This plays the same role as the bandwidth in kernel smoothing:

- small $h \Rightarrow$ many bins $\Rightarrow$ rough fit (high variance),
- large $h \Rightarrow$ few bins $\Rightarrow$ smooth fit (high bias).

Thus the regressogram illustrates clearly the bias–variance trade-off.

7.3 Example: ethanol and NOx

The slides use the ethanol engine dataset, relating ethanol concentration (E) to nitrogen oxide emissions (NOx).

```
library(NSM3)
data("ethanol")

library(HoRM)
library(ggplot2)

p = regressogram(ethanol$E, ethanol$NOx,
                 nbins = 20,
                 show.bins = TRUE,
                 show.means = TRUE,
                 show.lines = TRUE,
                 x.lab = "E", y.lab = "NOx",
                 main = "NOx and ethanol engines metric") +
    theme(plot.title = element_text(hjust = 0.5))

p
```

Interpretation

- Each bin produces a local average NOx value.
- The fitted curve is a step function.
- Increasing the number of bins makes the curve more flexible.
- The regressogram is easy to understand but not very smooth.

Because of its discontinuity, the regressogram is mainly a teaching tool. More refined smoothers, such as kernel regression and local polynomials, provide smoother estimates.

8 Local Averaging (Friedman / Running Mean)

8.1 Idea

The regressogram produces a step-function fit because bins are disjoint. A natural refinement is to allow *overlapping neighborhoods*.

The guiding principle is:

> To estimate $r(\boldsymbol{x})$, average the responses Y_j observed at predictor values $\boldsymbol{x}_j$ that are close to $\boldsymbol{x}$.

One common version uses a fixed window width $h > 0$.
Define the local neighborhood around $\boldsymbol{x}$:

$$B_x = \{i : \ |\boldsymbol{x}_i - \boldsymbol{x}| \leq h\}, \qquad n_{\boldsymbol{x}} = \#B_{\boldsymbol{x}}.$$

Then the local averaging estimator is:

$$\widehat{r}_n(\boldsymbol{x}) = \frac{1}{n_{\boldsymbol{x}}} \sum_{i \in B_{\boldsymbol{x}}} Y_i.$$

This can be written in linear smoother form:

$$\widehat{r}_n(\boldsymbol{x}) = \sum_{i=1}^{n} \ell_i(\boldsymbol{x}) Y_i,$$

where the weights are:

$$\ell_i(\boldsymbol{x}) = \begin{cases} \frac{1}{n_{\boldsymbol{x}}}, & i \in B_{\boldsymbol{x}}, \\ 0, & \text{otherwise.} \end{cases}$$

Interpretation

- Only observations within distance h of x contribute.
- All points in the window receive equal weight.
- The estimate is a moving average of nearby responses.

Role of the bandwidth h

The window width h controls smoothness:

- Small h: very local fit, low bias but high variance (wiggly curve).
- Large h: more averaging, higher bias but lower variance (smooth curve).

Thus local averaging provides the simplest illustration of the bias–variance trade-off.

Smoothing matrix viewpoint

Because neighborhoods overlap, the smoothing matrix L is no longer block diagonal (as in the regressogram). Each fitted value uses information from multiple nearby bins.

8.2 R illustration

Friedman's SuperSmoother (`supsmu`) function implements a variable-span running mean smoother, with automatic bandwidth selection by cross-validation.

```
fit.local.avg = supsmu(ethanol$E, ethanol$NOx, span = "cv")

df.fit.local.avg = data.frame(
  E.fit   = fit.local.avg$x,
  NOx.fit = fit.local.avg$y
)

library(ggplot2)

p = ggplot() +
```

```
  geom_point(data = ethanol,
             aes(x = E, y = NOx)) +
  geom_line(data = df.fit.local.avg,
            aes(x = E.fit, y = NOx.fit),
            color = "red")

p
```

This produces a smooth curve without explicitly choosing h.

Remark: Running mean smoothers are intuitive, but equal weighting and boundary problems motivate more advanced approaches.

9 Local Regression and Local Polynomial Regression

9.1 Motivation

Local averaging uses constant fits inside neighborhoods. A natural improvement is:

> Instead of fitting a constant locally, fit a line (or polynomial) locally.

This leads to local regression methods developed by Cleveland, implemented in `loess`.

Advantages:

- smoother curves,
- better behavior near boundaries,
- reduced bias compared to local averaging.

9.2 Local linear regression

To estimate $r(x)$, fit a weighted least squares line around x.

For a target point x:

$$(\hat{\beta}_0(x), \hat{\beta}_1(x)) = \arg\min_{\beta_0,\beta_1} \sum_{j=1}^{n} w_j(x) \left(Y_j - \beta_0 - \beta_1(x_j - x)\right)^2.$$

Then the regression estimate is:

$$\widehat{r}_n(x) = \hat{\beta}_0(x).$$

Interpretation

- The slope $\hat{\beta}_1(x)$ captures local trend.
- The intercept $\hat{\beta}_0(x)$ gives the fitted value at x.
- Points closer to x receive higher weight.

Weight function

The weights are typically kernel-like:

$$w_j(x) = W\left(\frac{|x_j - x|}{h}\right),$$

where W decreases smoothly with distance.
The default in `loess` is the tricube weight:

$$W(u) = (1 - u^3)^3 \mathbf{1}(u < 1).$$

9.3 R illustration

```
fit.local.lin.reg = loess(NOx ~ E,
                          data = ethanol,
                          degree = 1,
                          span = 0.19)

df.fit.local.lin.reg = data.frame(
  E       = ethanol$E,
  NOx.fit = fit.local.lin.reg$fitted
)

p = ggplot() +
  geom_point(data = ethanol,
             aes(x = E, y = NOx)) +
  geom_line(data = df.fit.local.lin.reg,
            aes(x = E, y = NOx.fit),
            color = "red")

p
```

Local linear regression corrects much of the boundary bias seen in kernel regression.

9.4 Local polynomial regression

Local linear regression can be generalized by fitting a polynomial of degree d:

$$(\hat{\beta}_0(x), \ldots, \hat{\beta}_d(x)) = \arg\min_{\beta_0,\ldots,\beta_d} \sum_{j=1}^{n} w_j(x)\Big(Y_j - \sum_{k=0}^{d} \beta_k (x_j - x)^k\Big)^2.$$

The fitted value remains:

$$\widehat{r}_n(x) = \hat{\beta}_0(x).$$

Practical notes

- $d = 0$ gives local constant regression (kernel regression).
- $d = 1$ gives local linear regression.
- $d = 2$ gives local quadratic regression.

- Higher degrees are rarely needed in practice.

9.5 R illustration (quadratic local polynomial)

```
fit.local.poly.reg = loess(NOx ~ E,
                           data = ethanol,
                           degree = 2,
                           span = 0.2)

df.fit.local.poly.reg = data.frame(
  E       = ethanol$E,
  NOx.fit = fit.local.poly.reg$fitted
)

p = ggplot() +
  geom_point(data = ethanol,
             aes(x = E, y = NOx)) +
  geom_line(data = df.fit.local.poly.reg,
            aes(x = E, y = NOx.fit),
            color = "red")

p
```

10 Kernel Regression (Nadaraya-Watson)

10.1 Definition

Kernel regression estimates $r(x)$ as a kernel-weighted average:

$$\widehat{r}_n(x) = \sum_{i=1}^{n} \ell_i(x) Y_i,$$

with weights:

$$\ell_i(x) = \frac{K\left(\frac{x-\boldsymbol{x}_i}{h}\right)}{\sum_{j=1}^{n} K\left(\frac{x-x_j}{h}\right)}.$$

Thus:

- points closer to x receive larger weight,
- weights sum to 1, ensuring constant reproduction.

Key messages

- The choice of bandwidth h is much more important than kernel choice.
- Small h produces undersmoothing (wiggly curve).
- Large h produces oversmoothing (high bias).
- Kernel regression suffers from boundary bias near the edges.
- Local polynomial regression helps alleviate boundary bias.

10.2 Bandwidth selection and risk

The optimal bandwidth depends on unknown smoothness properties of $r(x)$.

In general:

$$\mathrm{MSE}(\widehat{r}_n(x)) \approx \underbrace{C_1 h^{2s}}_{\text{bias}^2} + \underbrace{\frac{C_2}{nh}}_{\text{variance}},$$

where s is the smoothness level of r.

Thus:

- bias increases with h,
- variance decreases with h,
- cross-validation is commonly used to choose $h^\star$.

10.3 R illustration (package `np`)

```
library(np)

ethanol.npreg  <- npreg(bws = .09,
                        txdat = ethanol$E,
                        tydat = ethanol$NOx,
                        ckertype = "epanechnikov")

ethanol.npreg2 <- npreg(bws = .03,
                        txdat = ethanol$E,
                        tydat = ethanol$NOx,
                        ckertype = "epanechnikov")

ethanol.npreg$MSE
ethanol.npreg2$MSE
```

Plotting two bandwidth fits

```
ethanol.npreg.fit = data.frame(
  E = ethanol$E,
  NOx = ethanol$NOx,
  kernel.fit  = fitted(ethanol.npreg),
  kernel.fit2 = fitted(ethanol.npreg2)
)

ggplot(ethanol.npreg.fit) +
  geom_point(aes(x = E, y = NOx)) +
  geom_line(aes(x = E, y = kernel.fit),  color = "red") +
  geom_line(aes(x = E, y = kernel.fit2), color = "blue")
```

This plot illustrates clearly how bandwidth controls smoothness.

Brief proof. Estimate the regression function

$$m(x) := \mathbb{E}[Y \mid X = x].$$

Start from conditional expectation. Using the definition of conditional expectation,

$$\mathbb{E}[Y \mid X = x] = \int y\, f_{Y|X}(y \mid x)\, dy = \int y\, \frac{f_{X,Y}(x,y)}{f_X(x)}\, dy.$$

Plug-in KDEs for the densities. Estimate the joint density and the marginal density by kernel density estimation (with a kernel $K_h(\cdot)$ and bandwidth h):

$$\widehat{f}_{X,Y}(x,y) = \frac{1}{n}\sum_{i=1}^{n} K_h(x - \boldsymbol{x}_i)\, K_h(y - y_i), \qquad \widehat{f}_X(x) = \frac{1}{n}\sum_{i=1}^{n} K_h(x - \boldsymbol{x}_i).$$

Plug-in estimator of the conditional mean. Define

$$\widehat{\mathbb{E}}[Y \mid X = x] := \int y\, \frac{\widehat{f}_{X,Y}(x,y)}{\widehat{f}_X(x)}\, dy.$$

Then

$$\begin{aligned}
\widehat{\mathbb{E}}[Y \mid X = x] &= \int y\, \frac{\sum_{i=1}^{n} K_h(x - \boldsymbol{x}_i)\, K_h(y - y_i)}{\sum_{j=1}^{n} K_h(x - x_j)}\, dy \\
&= \frac{\sum_{i=1}^{n} K_h(x - \boldsymbol{x}_i) \int y\, K_h(y - y_i)\, dy}{\sum_{j=1}^{n} K_h(x - x_j)} \\
&= \frac{\sum_{i=1}^{n} K_h(x - \boldsymbol{x}_i)\, y_i}{\sum_{j=1}^{n} K_h(x - x_j)}.
\end{aligned}$$

This is the *Nadaraya–Watson kernel regression* estimator:

$$\widehat{m}_h(x) = \frac{\sum_{i=1}^{n} K_h(x - \boldsymbol{x}_i)\, y_i}{\sum_{i=1}^{n} K_h(x - \boldsymbol{x}_i)}.$$

11 Penalized Regression and Smoothing Splines

11.1 Penalized least squares: the big idea

Up to now, we have viewed smoothing as *local fitting*: averaging nearby points, fitting local lines, or weighting observations by distance.

An alternative and very powerful viewpoint is **penalized regression**.

The idea is to estimate $r(\cdot)$ by balancing:

- goodness of fit to the data, and
- smoothness of the fitted function.

This leads to the penalized least squares criterion:

$$M(\lambda) = \sum_{i=1}^{n} \big(Y_i - r(\boldsymbol{x}_i)\big)^2 \;+\; \lambda\, J(r),$$

where:

- the first term measures fidelity to the data,
- $J(r)$ is a roughness penalty,
- $\lambda \geq 0$ is the smoothing parameter.

The classical and most important choice is:

$$J(r) = \int \{r''(x)\}^2 \, dx,$$

which penalizes curvature.

Interpretation of the penalty

- Large $|r''(x)|$ means the function bends sharply.
- Integrating $(r'')^2$ measures total curvature.
- Minimizing $J(r)$ favors smooth, slowly varying functions.

Thus, penalized regression explicitly encodes the notion of smoothness.

11.2 Role of the smoothing parameter λ

The parameter λ controls the trade-off between fit and smoothness:

- $\lambda = 0$: no penalty; the solution interpolates the data (passes exactly through all $(\boldsymbol{x}_i, Y_i)$, very wiggly).
- Small λ: close to interpolation, low bias but high variance.
- Large λ: strong penalty; the curve becomes very smooth.
- As $\lambda \to \infty$: the solution approaches the least squares line.

Thus λ plays the same conceptual role as the bandwidth h in kernel smoothing.

Bandwidth h and penalty λ are two ways of controlling smoothness.

11.3 Splines and smoothing splines

To solve the penalized problem, we need to restrict attention to a manageable class of functions. This leads naturally to **splines**.

Cubic splines

A cubic spline is a function that:

- is a cubic polynomial between consecutive knots,
- is continuous at the knots,
- has continuous first and second derivatives at the knots.

Cubic splines provide enough flexibility to model nonlinear trends while retaining smoothness.

Smoothing splines

A key theoretical result (from calculus of variations) is:

> The minimizer of
> $$\sum_{i=1}^{n}(Y_i - r(x_i))^2 + \lambda \int (r''(x))^2 dx$$
> over all twice-differentiable functions r is a *natural cubic spline* with knots at the observed x_i.

Thus:

- smoothing splines arise naturally from penalized least squares,
- no knot selection is needed (knots are at the data points),
- smoothness is controlled entirely by λ.

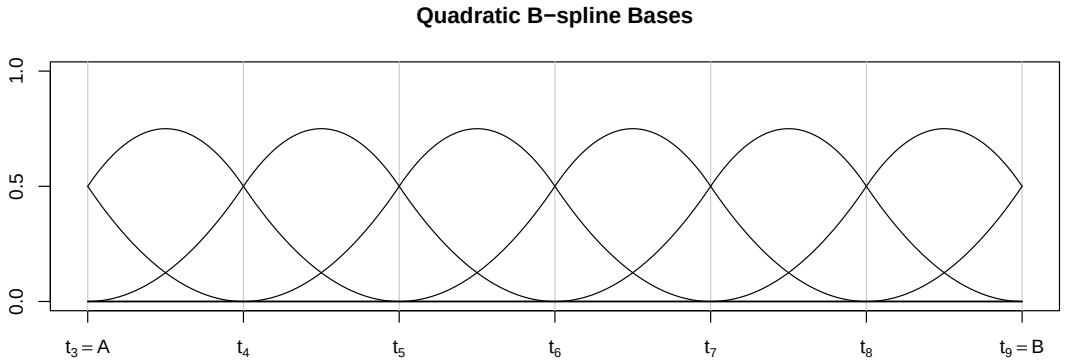

Figure 2: Quadratic B-splines defined with 11 equidistant knot-points partitioning the interval $[A, B]$ into 6 equal sub-intervals.

11.4 Basis expansion and ridge-like form

Although smoothing splines are defined functionally, they can be computed using basis expansions.

Let $\{B_1(x), \ldots, B_N(x)\}$ be a spline basis (e.g., B-splines). Then write:

$$\widehat{r}_n(x) = \sum_{j=1}^{N} \beta_j B_j(x).$$

Plugging this into the penalized criterion yields:

$$\sum_{i=1}^{n}\Big(Y_i - \sum_{j=1}^{N} \beta_j B_j(x_i)\Big)^2 \; + \; \lambda\, \boldsymbol{\beta}^\top \Omega \boldsymbol{\beta},$$

where Ω is a penalty matrix derived from $\int (r'')^2$.

The solution has a familiar form:

$$\hat{\boldsymbol{\beta}} = (B^\top B + \lambda\Omega)^{-1} B^\top \boldsymbol{Y},$$

and the fitted values are:

$$\hat{\mathbf{r}} = B\hat{\boldsymbol{\beta}} = L\boldsymbol{Y}.$$

Key insight

- Smoothing splines are **linear smoothers**.
- The smoothing matrix L depends on λ.
- This is analogous to ridge regression, but with a structured penalty.

Thus smoothing splines unify:

- nonparametric regression,
- penalized regression,
- linear smoother theory.

11.5 R illustration: cubic spline regression (no penalty)

A cubic spline with fixed knots can be fit using basis functions.

```
library(splines)

range(ethanol$E)

cubic.spline.fit = lm(NOx ~ bs(E, knots = c(.75, 1, 1.2)),
                      data = ethanol)

summary(cubic.spline.fit)

df.cubic.spline = data.frame(
  x   = ethanol$E,
  y   = ethanol$NOx,
  fit = fitted(cubic.spline.fit)
)

library(ggplot2)

p = ggplot(data = df.cubic.spline) +
  geom_point(aes(x = x, y = y)) +
  geom_line(aes(x = x, y = fit), color = "red") +
  geom_vline(xintercept = c(.75, 1, 1.2), color = "green")

p
```

This fit depends critically on knot placement and has no explicit smoothness penalty.

11.6 R illustration: smoothing spline (penalized, CV-selected)

Smoothing splines automate smoothness selection via cross-validation.

```
smooth.spline.fit =
  smooth.spline(ethanol$E, ethanol$NOx, cv = TRUE)

df.smooth.splines = data.frame(
  x = smooth.spline.fit$x,
  fit.smooth.spline = smooth.spline.fit$y
)

p = ggplot() +
  geom_point(data = df.cubic.spline, aes(x = x, y = y)) +
  geom_line(data = df.cubic.spline,
            aes(x = x, y = fit),
            color = "red") +
  geom_line(data = df.smooth.splines,
            aes(x = x, y = fit.smooth.spline),
            color = "brown")

p
```

Interpretation

- The smoothing spline adapts automatically to the data.
- Smoothness is controlled by λ, chosen by CV.
- No manual knot selection is required.

11.7 Additive models and GAM regression

Smoothing splines handle nonlinear relationships between a response Y and a *single* predictor X. In practice, however, we often have several predictors:

$$Y \text{ depends on } X_1, X_2, \ldots, X_p.$$

A fully nonparametric regression model would estimate

$$r(x_1, \ldots, x_p) = \mathbb{E}(Y \mid X_1 = x_1, \ldots, X_p = x_p),$$

but this suffers severely from the curse of dimensionality.

A practical compromise is the **additive model**:

$$\mathbb{E}(Y \mid X_1, \ldots, X_p) = \beta_0 + r_1(X_1) + r_2(X_2) + \cdots + r_p(X_p),$$

where each $r_j(\cdot)$ is a smooth function, typically modelled using spline bases.

This model is called a **Generalized Additive Model (GAM)**.
Some great references:

- `https://quantling.org/~hbaayen/publications/BaayenLinke2020.pdf`
- Book: Wood, S. N. (2017). Generalized additive models: an introduction with R. chapman and hall/CRC.

Why additive models?

Additive models provide a balance between flexibility and interpretability.

- Each predictor has its own smooth effect.
- Nonlinear relationships can be captured.
- The model avoids high-dimensional smoothing.
- Each function r_j can be visualized separately.

Thus additive models reduce the multivariate problem into a collection of one-dimensional smoothing problems.

Penalized formulation

Each component function is estimated using penalized regression:

$$\sum_{i=1}^{n}\left(Y_i-\beta_0-\sum_{j=1}^{p}r_j(X_{ij})\right)^2+\sum_{j=1}^{p}\lambda_j\int\{r_j''(x)\}^2dx.$$

Here:

- Each r_j is a smooth function.
- Each λ_j controls smoothness.
- The penalties prevent overfitting.

Thus GAM regression is a direct extension of smoothing splines to multiple predictors.

Basis expansion view

Each smooth function can be written using a spline basis:

$$r_j(x)=\sum_{k=1}^{K_j}\beta_{jk}B_{jk}(x).$$

Then the model becomes

$$Y_i=\beta_0+\sum_{j=1}^{p}\sum_{k=1}^{K_j}\beta_{jk}B_{jk}(X_{ij})+\varepsilon_i.$$

Penalties of the form

$$\lambda_j\boldsymbol{\beta}_j^\top\Omega_j\boldsymbol{\beta}_j$$

control smoothness for each predictor.

Thus GAM estimation reduces to a penalized regression problem, similar to smoothing splines but with multiple smooth components.

11.7.1 R illustration: GAM regression

Generalized additive models can be fit using the `mgcv` package.

```
library(mgcv)

gam.fit =
  gam(NOx ~ s(E) + C,
      data = ethanol)

summary(gam.fit)
```

Visualization

```
plot(gam.fit, pages = 1)
```

Interpretation

- Each term `s(X)` represents a smooth function.
- Smoothness is controlled automatically by penalization.
- The model estimates separate curves:

$$r_1(E), \qquad r_2(C).$$

- The fitted model is

$$\mathbb{E}(Y \mid E, C) = \beta_0 + r_1(E) + r_2(C).$$

Key insight:

- Smoothing splines estimate one function.
- GAMs estimate several functions simultaneously.
- Both are based on penalized least squares.

Thus generalized additive models extend smoothing splines to multivariate regression while retaining interpretability and computational tractability. It also has the mixed model version for repeated or longitudinal outcomes called GAMM, which is also available in `mgcv`.

12 Choosing a Smoother: Practical Guidance

The lectures emphasize that there is no universally best smoother. The choice depends on:

- **Bias–variance trade-off:** how smooth should the curve be?
- **Computational cost:** splines and kernels scale differently with n.

- **Boundary behavior:** kernel methods suffer boundary bias; local polynomials help.
- **Dimensionality:** nonparametric methods degrade rapidly in high dimension.
- **Inferential goals:** prediction vs interpretation; confidence bands require additional theory.

Unifying message:

> All smoothers trade bias for variance. They differ mainly in how this trade-off is implemented and controlled.

What does "nonparametric" mean here?

In nonparametric regression the goal is to estimate the regression function

$$m_0(x) = \mathbb{E}[Y \mid X = x]$$

without assuming a specific parametric form (such as linear or polynomial). Thus, m_0 is allowed to be an arbitrary smooth function.

In practice, m_0 may be approximated by a finite expansion, for example using spline basis functions:

$$\hat{m}(x) = \sum_{j=1}^{p} \hat{\beta}_j \, g_j(x),$$

where $g_1, \ldots, g_p$ are spline basis functions and $\hat{\beta}_1, \ldots, \hat{\beta}_p$ are estimated coefficients.

Although the coefficients $\beta_1, \ldots, \beta_p$ are parameters, the model is still called *nonparametric* because we do not assume that

$$m_0(x) = \sum_{j=1}^{p} \beta_j g_j(x)$$

holds exactly. Instead, the basis expansion is only an approximation, and the flexibility of the estimator typically increases with the sample size (for example by allowing $p = p_n \to \infty$).

13 References (as listed in the slides)

- HWC Chapter 12 (density estimation) and Chapter 14 (smoothing / nonparametric regression).
- Wasserman (2006), Chapters 4–5 (smoothing concepts; linear smoothers).
- Seiler (2016), lecture notes (as referenced in the slides).
- Hastie, Tibshirani, Friedman (2009) for boundary-bias illustrations and smoothing context.